Backend Specification: Distributed Video Meeting System

Status: Draft v1 · **Owner:** Backend Platform · **Last updated:** 2026-09-01

---

1. Overview

This document specifies the backend for a real-time video meeting platform comparable to Google Meet or Zoom. It covers signaling, media routing, meeting state, recording, and the supporting control plane. It does not cover client UI or codec internals.

1.1 Goals

- Multi-party audio/video meetings up to 500 participants (250 with video, remainder audio/view-only).
- Sub-200 ms mouth-to-ear latency for participants in the same region; < 400 ms cross-region.
- 99.95% availability for meeting join; 99.99% for in-progress media once joined.
- Screen sharing, chat, reactions, hand-raise, host controls, breakout rooms.
- Cloud recording with transcript.
- Horizontal scalability with no single global coordinator on the media path.

1.2 Non-goals (v1)

- End-to-end encryption with client-managed keys (v2 — design leaves room for it).
- Telephony dial-in (PSTN gateway is a separate spec).
- Live streaming to third-party platforms (RTMP egress) beyond a stub.
- Meetings > 1,000 participants (webinar mode).

1.3 Assumptions

- Clients are browsers (WebRTC) and native apps using a WebRTC stack.
- Media is routed through Selective Forwarding Units (SFUs), not mixed (MCU), except for recording composition.
- Deployment target is Kubernetes on at least three cloud regions with anycast/GeoDNS in front.
- Team: ~12 backend engineers, 12-month horizon to GA.

---

2. Requirements

2.1 Functional

ID	Requirement
F1	Create scheduled or instant meetings; generate a human-typeable meeting code (e.g. `abc-defg-hij`).
F2	Join by code or link; support lobby/waiting room and host admission.
F3	Publish/subscribe audio, video, and screen-share tracks; simulcast and adaptive subscription.
F4	Mute/unmute self; host can mute others, remove participants, lock meeting, end for all.
F5	In-meeting chat, emoji reactions, hand raise, polls (basic).
F6	Breakout rooms: host creates N rooms, assigns participants, recalls.
F7	Cloud recording (composited MP4 + per-track archive) with automatic transcription.
F8	Active-speaker detection and dominant-speaker events.
F9	Reconnection within 30 s preserves participant identity and state.
F10	Calendar integration and meeting metadata (title, organizer, attendees).
F11	Enterprise controls: SSO, domain restrictions, data residency region pinning.

2.2 Non-functional

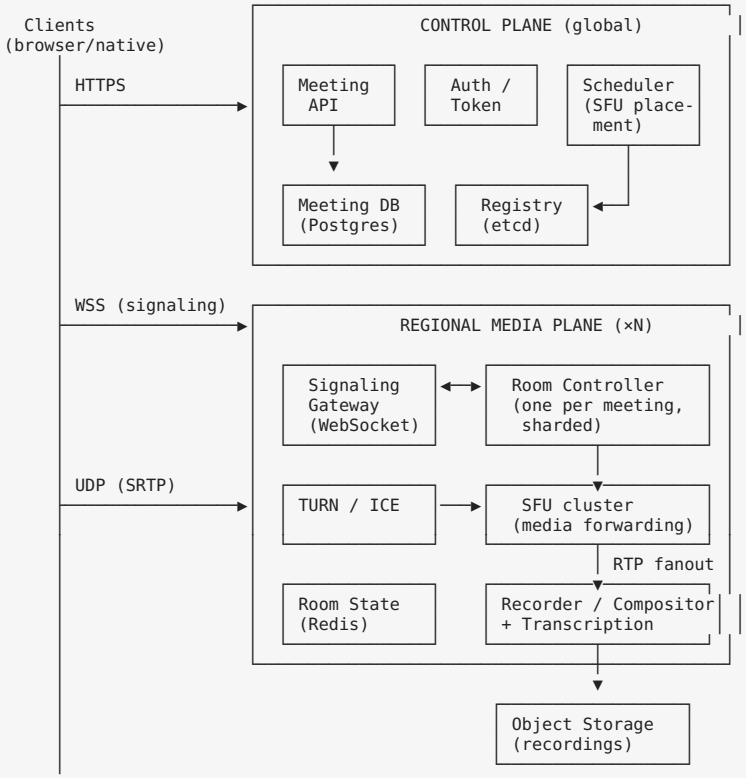
Category	Target
Join latency (p95)	< 2 s from click to first frame received
Media latency (p95)	< 200 ms intra-region, < 400 ms inter-region
Availability	99.95% control plane, 99.99% media plane (per-meeting)
Scale	100k concurrent meetings, 1M concurrent participants, 10k joins/s peak
Packet loss tolerance	Usable at 20% loss (via FEC, NACK, RED, simulcast downgrade)
Durability	Recordings: 11 nines (object storage); meeting metadata: no loss on single-AZ failure
Security	DTLS-SRTP on all media; TLS 1.3 on signaling; per-meeting tokens with 5-min TTL
Data residency	Meeting media and recordings stay in the organizer's configured region

2.3 Load estimation

- 1M participants × avg 3 subscribed video streams × ~600 kbps ≈ 1.8 Tbps aggregate SFU egress.
- Single SFU node (16 vCPU, 25 Gbps NIC) handles ~1,500 participant sessions or ~8 Gbps egress; plan ~250 SFU nodes at peak, ~40% headroom.
- Signaling: ~20 messages/s per meeting average → 2M msgs/s across the fleet; trivial per node, dominated by connection count (1M long-lived WebSockets ≈ 100 gateway pods at 10k conns each).
- Meeting metadata store: ~100k writes/s peak during join storms (top of the hour). Design for 5× burst.

---

3. High-Level Architecture



3.1 Component summary

Component	Responsibility	Stateful?
Meeting API	CRUD for meetings, join authorization, issues join tokens	No
Auth / Token service	User auth (OIDC), signs short-lived JWTs for signaling and media	No
Scheduler	Picks region + SFU node for a new meeting; handles rebalancing	No (reads Registry)
Registry	Live inventory of SFU nodes, load, and meeting→SFU mapping	Yes (etcd)
Signaling Gateway	Terminates WebSockets, routes messages to the correct Room Controller	Connection state only
Room Controller	Authoritative in-memory state machine for one meeting: roster, permissions, track topology	Yes (checkpointed to Redis)
SFU	Receives RTP from publishers, forwards selectively to subscribers; simulcast layer selection; bandwidth estimation	Session state only
TURN	NAT traversal relay for clients that can't reach the SFU directly	No
Recorder	Subscribes to the SFU like a headless participant; writes raw tracks and composited output	Job state
Transcription	Consumes audio from Recorder; produces timestamped transcript	Job state
Event bus	Kafka; all meeting lifecycle and participant events for analytics, billing, webhooks	Yes

3.2 Key design decisions

1. **SFU over MCU.** Forwarding preserves quality and scales linearly; clients handle layout. Trade-off: client CPU and downlink scale with subscribed streams, mitigated by simulcast and subscription limits. MCU is used only for recording composition, where a single mixed output is needed.

2. **One Room Controller per meeting, sharded across regional pods.** Gives a single source of truth for room state without distributed consensus on the hot path. Trade-off: controller failover requires state recovery (see §7.2); mitigated

by continuous checkpointing to Redis.

3. **Cascaded SFUs for cross-region meetings.** A meeting has one *home* SFU; participants in other regions connect to a *local* SFU which peers with the home SFU over a single RTP trunk per track. Trade-off: added complexity and one extra hop of latency (~30–80 ms) for remote participants, versus every remote participant hairpinning across regions.
4. **Signaling decoupled from media.** WebSocket gateway is stateless and can be replaced without dropping media. Media continues during signaling reconnect.
5. **Meeting state in memory, checkpointed to Redis; durable metadata in Postgres.** Postgres is never on the in-meeting hot path.

---

4. Data Model

4.1 Postgres (durable, control plane)

```
CREATE TABLE meetings (
  id              UUID PRIMARY KEY,
  code            TEXT UNIQUE NOT NULL,        -- 'abc-defg-hij'
  org_id          UUID NOT NULL,
  organizer_id    UUID NOT NULL,
  title           TEXT,
  scheduled_start TIMESTAMPTZ,
  scheduled_end   TIMESTAMPTZ,
  region          TEXT NOT NULL,               -- residency pin, e.g. 'us-west'
  settings        JSONB NOT NULL,              -- lobby, recording policy, chat, etc.
  status          TEXT NOT NULL,               -- scheduled | live | ended | cancelled
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  ended_at        TIMESTAMPTZ
);
CREATE INDEX ON meetings (org_id, scheduled_start);

CREATE TABLE meeting_sessions (               -- one row per "live" instance of a meeting
  id            UUID PRIMARY KEY,
  meeting_id    UUID REFERENCES meetings(id),
  started_at    TIMESTAMPTZ NOT NULL,
  ended_at      TIMESTAMPTZ,
  home_sfu_id   TEXT,
  peak_participants INT
);

CREATE TABLE participants (                   -- durable attendance record
  id            UUID PRIMARY KEY,
  session_id    UUID REFERENCES meeting_sessions(id),
  user_id       UUID,                         -- NULL for guests
  display_name  TEXT NOT NULL,
  role          TEXT NOT NULL,                -- host | cohost | attendee
  joined_at     TIMESTAMPTZ NOT NULL,
  left_at       TIMESTAMPTZ,
  leave_reason  TEXT
);

CREATE TABLE recordings (
  id            UUID PRIMARY KEY,
  session_id    UUID REFERENCES meeting_sessions(id),
  status        TEXT NOT NULL,                -- recording | processing | ready | failed
  storage_uri   TEXT,
  duration_s    INT,
  transcript_uri TEXT,
  created_at    TIMESTAMPTZ NOT NULL
);

CREATE TABLE chat_messages (                  -- persisted only if org policy allows
  id          UUID PRIMARY KEY,
  session_id  UUID NOT NULL,
  sender_id   UUID NOT NULL,
  sent_at     TIMESTAMPTZ NOT NULL,
  body        TEXT NOT NULL
);
```

4.2 Redis (room state checkpoint, per region)

Key layout, all under a hash-tag `{room:<session_id>}` so they co-locate in one cluster slot:

```
{room:S}:meta          HASH   status, home_sfu, host_ids, locked, lobby_enabled, recording
{room:S}:roster        HASH   participant_id -> JSON {name, role, muted, video, hand, sfu, conn_id}
{room:S}:lobby         HASH   participant_id -> JSON {name, requested_at}
{room:S}:tracks        HASH   track_id -> JSON {owner, kind, simulcast_layers, paused}
{room:S}:breakouts     HASH   room_idx -> JSON {name, member_ids}
{room:S}:seq           STRING monotonic event sequence for client resync
```

TTL of 1 h refreshed by the Room Controller heartbeat; expiry implies the meeting is dead.

4.3 etcd (registry)

```
/sfu/<region>/<node_id>        {addr, capacity, load, version, healthy, lease}
/meeting/<session_id>          {home_region, home_sfu, controller_pod, cascades: [...]}
/controller/<region>/<pod_id>  {lease, room_count}
```

4.4 Kafka topics

Topic	Key	Producer	Consumers
`meeting.lifecycle`	session_id	Room Controller	Analytics, Billing, Webhooks, Postgres writer
`participant.events`	session_id	Room Controller	Analytics, QoS dashboards
`media.qos`	session_id	SFU (1 Hz sampled)	QoS pipeline, alerting
`recording.jobs`	recording_id	Room Controller	Recorder, Transcriber

Partitioned by session_id so all events for one meeting are ordered.

---

5. API Contracts

5.1 REST — Meeting API (control plane)

All endpoints under `/v1`, authenticated via OIDC bearer token (or guest token for join).

```
POST   /meetings                       Create meeting
GET    /meetings/{id}                  Get meeting
PATCH  /meetings/{id}                  Update settings/schedule
DELETE /meetings/{id}                  Cancel
POST   /meetings/{id}:end              Host ends for all

POST   /join                           Resolve code → join grant
GET    /meetings/{id}/recordings       List recordings
GET    /recordings/{id}                Get recording + signed download URL
```

`POST /join` — the critical path.

Request:

```
{ "code": "abc-defg-hij", "display_name": "Ana", "client_caps": { "simulcast": true, "av1": false } }
```

Response `200`:

```
{
  "session_id": "…",
  "participant_id": "…",
  "role": "attendee",
  "lobby": false,
  "signaling_url": "wss://sig-usw2.example.com/rooms/{session_id}",
  "signaling_token": "<JWT, 5 min>",
  "ice_servers": [
    { "urls": "stun:stun.example.com:3478" },
    { "urls": "turn:turn-usw2.example.com:443?transport=tcp", "username": "…", "credential": "…" }
  ],
  "region": "us-west-2"
}
```

Behavior: the API looks up the meeting, enforces org policy (domain restriction, lock state), asks the Scheduler for a region/SFU (creating the session and Room Controller if the meeting isn't live), and mints a signaling JWT containing `{session_id,`

`participant_id, role, sfu_hint, exp}`. The response steers the client to the nearest signaling gateway that can reach that meeting.

Errors: `404 MEETING_NOT_FOUND`, `403 LOCKED`, `403 DOMAIN_RESTRICTED`, `410 ENDED`, `429 CAPACITY` (meeting full), `503 NO_CAPACITY` (regional, client should retry with backoff).

5.2 Signaling protocol (WebSocket, JSON)

Every message: `{ "type": "…", "seq": n, "id": "<client-generated for req/resp>", "payload": {…} }`. Server pushes carry a monotonic `seq` so clients can detect gaps and request resync.

Client → Server

Type	Payload	Notes
`hello`	`{token, resume_seq?}`	First message. `resume_seq` on reconnect triggers delta sync.
`publish`	`{track_id, kind: audio|video|screen, sdp_offer, simulcast: [...]}`	One offer per publish; server answers.
`unpublish`	`{track_id}`	
`subscribe`	`{track_ids: [...], prefs: {track_id: {max_layer, priority}}}`	Idempotent full subscription set.
`ice_candidate`	`{transport: pub|sub, candidate}`	Trickle ICE.
`set_media_state`	`{muted?, video_off?}`	
`raise_hand` / `lower_hand`	`{}`	
`reaction`	`{emoji}`	Rate-limited 2/s.
`chat`	`{body, to?: participant_id}`	Max 4 KB.
`host.admit` / `host.deny`	`{participant_id}`	Host/cohost only.
`host.mute` / `host.remove` / `host.lock` / `host.end`	varies	Host/cohost only.
`breakout.create` / `breakout.assign` / `breakout.close`	varies	Host only.
`recording.start` / `recording.stop`	`{}`	Per org policy.
`ping`	`{ts}`	Every 10 s; 3 missed → server closes.

Server → Client

Type	Payload
`welcome`	`{participant_id, roster, tracks, room_meta, seq}` — full snapshot
`resync`	Same as `welcome`; sent when `resume_seq` is too old
`sdp_answer`	`{transport, sdp}`
`ice_candidate`	`{transport, candidate}`
`participant.joined` / `.left` / `.updated`	roster delta
`track.added` / `.removed` / `.updated`	track delta (incl. layer availability)
`active_speakers`	`{participant_ids: [...]}` (every 300 ms while changing)
`lobby.request`	`{participant_id, name}` → hosts only
`lobby.result`	`{admitted: bool}` → the waiting participant
`chat` / `reaction` / `hand`	fan-out
`breakout.moved`	`{room_idx, signaling_url, token}` — client reconnects to new room
`room.ended`	`{reason}`
`error`	`{id, code, message}`

Design notes

- Two peer connections per client: one send (publisher), one receive (subscriber). Avoids renegotiation storms; the subscriber PC uses a single transceiver per subscribed track with server-side `replaceTrack` semantics.
- `subscribe` is declarative (full set), so client and server can never disagree on the intended topology after a reconnect.
- Server never trusts client-reported role; role is in the JWT and checked per message.

5.3 Internal gRPC — Room Controller ↔ SFU

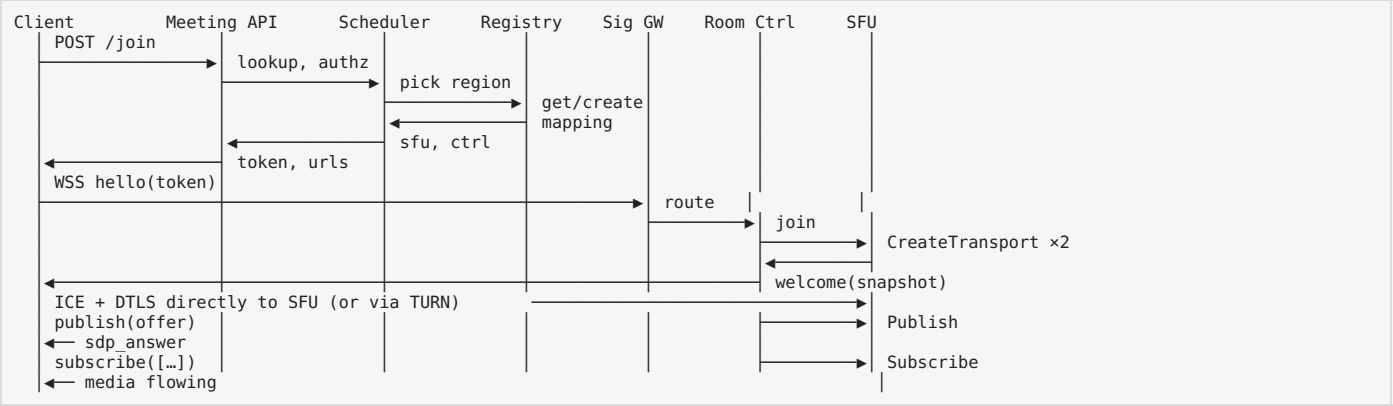
```
service Sfu {
  rpc AllocateRoom(AllocateRoomReq)   returns (AllocateRoomResp);
  rpc CreateTransport(TransportReq)   returns (TransportResp);   // returns ICE/DTLS params
  rpc Publish(PublishReq)             returns (PublishResp);
  rpc Subscribe(SubscribeReq)         returns (SubscribeResp);
  rpc SetLayer(SetLayerReq)           returns (Empty);
  rpc Cascade(CascadeReq)             returns (CascadeResp);     // trunk tracks to remote SFU
  rpc Stats(StatsReq)                 returns (stream StatsEvent);// active speaker, BWE, loss
  rpc CloseRoom(CloseRoomReq)         returns (Empty);
}
```

The Room Controller owns *what* should be forwarded to whom; the SFU owns *how* (layer choice within the allowed max, congestion control).

---

6. Core Flows

6.1 Join



Target: `/join` < 150 ms p95; WebSocket + ICE + first frame < 2 s p95. ICE is the long pole; TURN credentials are pre-issued in `/join` so the client can gather relay candidates in parallel with signaling.

6.2 Adaptive subscription

1. Publisher sends simulcast: 180p@150k, 360p@500k, 720p@1.5M (video); screen share sends 2 layers (low-fps thumbnail, full).
2. Client declares per-track preferences (`max_layer`, `priority`) based on its layout — tile size and visibility.
3. SFU runs transport-wide congestion control (TWCC) per subscriber; when estimated bandwidth drops, it downgrades layers starting with lowest-priority tracks, then pauses video on the lowest priority, before ever touching audio.
4. Publisher-side: SFU sends REMB/TWCC feedback to the publisher; publisher drops top simulcast layer under uplink constraint. Room Controller receives `track.updated` with available layers and fans it out so clients can adjust UI.

6.3 Active speaker

SFU computes audio level from RTP header extension (`ssrc-audio-level`) per publisher, applies a hysteresis window (300 ms rise / 1 s fall), streams `StatsEvent{dominant_speakers}` to the Room Controller, which broadcasts `active_speakers`. Clients use it to auto-subscribe to the speaker's video at a higher layer.

6.4 Cross-region cascade

When a participant joins from region B for a meeting homed in region A:

1. Scheduler sees home is A, picks a local SFU in B, records the cascade in the Registry.
2. Room Controller (in A) calls `Cascade` on both SFUs to establish a trunk: each published track is forwarded once over the trunk (highest layer only; the remote SFU performs layer selection for its own subscribers).
3. The participant connects to SFU-B for media, but signaling still terminates at the single Room Controller in A (through any gateway). Signaling round-trip is cross-region but tolerable (~100 ms); media is local.

Cascade limit: 3 regions per meeting in v1; beyond that, remote participants connect directly to the home SFU.

6.5 Recording

1. Host sends `recording.start`. Room Controller validates policy, publishes `recording.jobs {start}`, emits `room.updated {recording: true}` (all clients show indicator — legal requirement in many jurisdictions).
2. Recorder pod claims the job, joins the meeting as a hidden participant via the internal gRPC path (no WebSocket), subscribes to all tracks at their top layer.
3. Writes per-track archives (Opus / VP8-VP9 in WebM segments, 10 s chunks) directly to object storage in the residency region as they arrive — the raw recording survives a Recorder crash.
4. Compositor (separate job, after `recording.stop`) renders a gallery/speaker layout to MP4 using the timestamped segments and the `active_speakers` event log. Transcriber runs in parallel on the mixed audio; output is WebVTT + JSON with speaker labels from track ownership.
5. `recordings.status → ready`; webhook emitted; signed URLs served through the Meeting API.

6.6 Breakout rooms

Each breakout is a child session (`parent_session_id`) with its own Room Controller and SFU allocation, preferring the same SFU

node so participants keep their existing transports where possible. `breakout.moved` tells the client to reconnect signaling to the child; media transport is reused if the SFU is the same, otherwise the client performs a full ICE restart. Recall reverses the process. Host retains a signaling session to the parent and can broadcast to all children.

---

7. Reliability

7.1 Failure modes and handling

Failure	Detection	Handling	Participant impact
Signaling GW pod dies	LB health check, client ping timeout	Client reconnects with `resume_seq`; any GW routes to the same Room Controller	None to media; roster updates delayed < 5 s
Room Controller pod dies	etcd lease expiry (10 s)	Regional supervisor reassigns rooms to another pod, which rehydrates from Redis checkpoint and re-attaches to SFU sessions via `Stats` stream	Host controls unavailable ~10–15 s; media unaffected
SFU node dies	Registry lease + SFU-side heartbeat to Controller (2 s)	Controller allocates room on new SFU, pushes `ice_restart` to all participants; publishers re-publish	~3–6 s media gap
Redis regional cluster down	Client error, circuit breaker	Controllers keep serving from memory; checkpoints buffered; new joins to *existing* rooms still work; controller failover degraded	None unless a controller also fails
Postgres down	Circuit breaker	New meeting creation fails (503); joins to already-live meetings succeed via Registry/Redis; events buffered in Kafka	Cannot create/schedule
Region isolated	GeoDNS health, cross-region probes	New meetings routed elsewhere; live meetings homed there continue for local participants; cascaded remote participants get `ice_restart` to home or re-home	Remote participants drop ~10 s
Kafka lag	Consumer lag alert	Non-critical path; nothing user-facing blocks on Kafka	Delayed analytics/webhooks

7.2 Room Controller checkpointing

Every state mutation is applied in memory then written to Redis in a pipelined MULTI (roster/track deltas + `seq`). Checkpoint is write-behind with a 50 ms coalescing window; worst-case data loss on crash is 50 ms of events, which is recovered by re-querying SFU session state on rehydration (the SFU is the truth for what is actually publishing).

7.3 Client reconnection contract

- Signaling: reconnect with exponential backoff (250 ms → 8 s), send `hello{resume_seq}`. Server replays deltas if `seq` is within the retained window (last 500 events or 60 s), else full `resync`.
- Media: ICE consent freshness detects dead paths within ~10 s; client triggers ICE restart on the same peer connection. Participant identity (`participant_id`) is stable across reconnects for 30 s grace; after that the participant is marked left and must rejoin via `/join`.

7.4 Capacity and admission control

- Scheduler placement: least-loaded SFU in the target region with ≥ 20% headroom; a meeting's projected size (from calendar attendee count) is reserved at creation.
- Per-meeting cap enforced by Room Controller; per-region cap enforced by Scheduler returning `503 NO_CAPACITY` with `Retry-After`.
- Load shedding order under regional saturation: reject new meetings → reject new joins to large meetings → cap max video layer at 360p fleet-wide → disable new recordings.

---

8. Security

- **Auth:** OIDC for users; guests get a signed guest token scoped to one meeting after lobby admission. All in-meeting authority derives from the signaling JWT (`role` claim); role changes (promote to cohost) issue a new token pushed via signaling.
- **Transport:** TLS 1.3 for HTTPS/WSS; DTLS-SRTP (AES-GCM) for all media. TURN over TLS on 443 for restrictive networks.
- **Meeting codes:** 10 alphanumeric chars ≈ 51 bits; brute-force protection via per-IP and per-code rate limits on `/join` (10/min), plus lobby by default for external participants.
- **Isolation:** SFU sessions are keyed by session_id; a transport cannot subscribe to tracks outside its room (enforced in SFU, not just controller).
- **Host controls:** Every `host.*` message re-validates role from the JWT; audit event emitted to `meeting.lifecycle`.
- **Recording consent:** Recording state is broadcast to all participants; the compositor stamps a recording notice in the first 5 s.
- **Data residency:** Region pin on the meeting row is enforced by the Scheduler (home SFU, Redis, Recorder, object storage bucket all region-local). Cascaded SFUs in other regions hold media transiently in memory only; this is documented as an exception in the residency policy.
- **E2EE readiness (v2):** Media path uses SFrame-compatible framing hooks; SFU never needs to decrypt payloads for forwarding. Active-speaker detection would move to client-reported levels.
- **Abuse:** Chat/reactions rate-limited; content moderation hook on chat via async pipeline; per-org allowlists.

---

9. Observability

9.1 Metrics (Prometheus)

Area	Key metrics	SLO alert
Join	`join_latency_seconds{p95}`, `join_errors_total{code}`	p95 > 2 s for 5 m; error rate > 1%
Media	`sfu_rtt_ms`, `sfu_packet_loss_ratio`, `sfu_jitter_ms`, `sfu_layer_downgrades_total`, per-subscriber BWE	p50 loss > 5% in a region
SFU capacity	`sfu_sessions`, `sfu_egress_bps`, `sfu_cpu`	> 80% for 10 m
Controllers	`room_count`, `checkpoint_lag_ms`, `failover_total`	checkpoint lag > 500 ms
Signaling	`ws_connections`, `ws_msg_latency_ms`, `resync_total`	resync rate spike
Recording	`recording_jobs{status}`, `segment_upload_lag_s`	any job stuck > 5 m

9.2 Per-meeting QoS record

SFU emits a `media.qos` event per participant every second (loss, RTT, jitter, sent/received layers, BWE). Stored in ClickHouse for 30 days; powers a "meeting health" view for support and drives automated regressions detection on SFU releases.

9.3 Tracing

OpenTelemetry spans across `/join` → Scheduler → Registry → Signaling → SFU `CreateTransport`, joined by `session_id` + `participant_id` baggage. Sampled 100% for errors, 1% otherwise.

---

10. Deployment

- **Regions:** ≥ 3 (e.g. us-west, eu-central, ap-southeast) at launch, each fully independent for the media plane; control plane is active-active with Postgres primary in one region and read replicas elsewhere (writes are off the hot path).
- **SFU nodes:** dedicated node pool, host networking, one SFU process per node, pinned CPU. Public IP per node for direct ICE; TURN as fallback. Nodes are cattle: drain by refusing new rooms, existing rooms migrate at end or after 4 h max age.
- **Room Controllers / Signaling GWs:** standard pods, HPA on connection count and CPU.
- **Releases:** SFU canaries take 2% of new rooms for 24 h with automated QoS comparison against baseline before promotion.
- **Config:** feature flags per org for lobby default, recording policy, max video layer.

---

11. Trade-offs and Alternatives Considered

Decision	Alternative	Why not
SFU	MCU (server-side mixing)	10–20× server CPU cost, adds latency, kills per-participant layout flexibility. Kept for recording only.
SFU	P2P mesh	Fine to 4 participants; uplink explodes beyond that.
Custom Room Controller state machine	Everything in Redis with Lua scripts	Higher per-message latency and harder to reason about invariants; in-memory + checkpoint is simpler and faster.
etcd Registry	Consul / DB table	etcd leases and watches map directly to node liveness needs; already present with Kubernetes.
Cascaded SFUs	Single home SFU for all participants	Cross-region hairpin adds 100–200 ms for everyone remote; cascade adds complexity but is standard practice at this scale.
WebSocket JSON signaling	gRPC-web / protobuf	Browser debuggability and ecosystem win at this message volume; can switch encoding later without changing message semantics.
Per-track WebM segments for recording	Record composited only	Raw tracks enable re-rendering layouts, per-speaker audio, and survive compositor bugs. Storage cost is modest.

---

12. What to Revisit as the System Grows

1. **> 1,000 participants:** introduce a webinar mode where attendees subscribe only, controllers shard the roster, and roster deltas are batched (1 Hz) instead of per-event.
2. **Signaling fan-out cost:** at 500 participants, one `participant.joined` becomes 500 WebSocket writes. Move to per-gateway multicast (one message per gateway, gateway fans out locally) before that becomes a problem.
3. **AV1 / SVC:** replace simulcast with scalable video coding once client support is broad; the SFU `SetLayer` abstraction already models spatial/temporal layers.
4. **E2EE:** promote from readiness hooks to full SFrame implementation; recording then requires a client-side or consented key-escrow path.
5. **Global Room Controller placement:** if a meeting's participant center of mass moves regions, migrate the controller (and re-home the SFU) rather than pay cross-region signaling indefinitely.
6. **Postgres write volume:** attendance rows for 1M participants/hour are fine on a single primary; at 5× that, shard `participants` by `session_id` hash or move to a wide-column store.
7. **TURN cost:** monitor relay ratio; if > 15% of sessions use TURN, invest in more direct-path IPs and IPv6 rather than more relay capacity.

Appendix A — Glossary

- **SFU** — Selective Forwarding Unit; routes RTP packets without decoding.
- **MCU** — Multipoint Control Unit; decodes, mixes, re-encodes.
- **Simulcast** — Publisher sends multiple resolutions of the same stream simultaneously.
- **TWCC / REMB** — Bandwidth-estimation feedback mechanisms in WebRTC.
- **ICE / TURN / STUN** — NAT traversal protocols; TURN relays media when direct paths fail.
- **DTLS-SRTP** — Key exchange and encryption for WebRTC media.
- **SFrame** — Frame-level encryption enabling E2EE through an SFU.